CSE 242 Assignment 1 Prompts

Ioannis Barous

Prompt 1

cse 242 hw1. attaching the starter file Assignment1.py. keep every signature and the comment block structure exactly as is, answers go in the # YOUR ANSWER HERE comments, code at # YOUR CODE HERE. functions can't mutate inputs or do io, plotting only under main. grader calls them with non-default args too so handle None defaults properly and don't hardcode the given numbers.

start with part 1. for Q1 write out AB, BA, (A+B)^T, AB^T by hand in the comments, plus trace(A) and trace(AB) (should be 4 and 6). for Q2 get the char poly of M, eigenvalues 1 and 3, eigenvectors [1,1] and [1,-1], and show Mv = λv for both. implement eigensystem with np.linalg.eigh since M is symmetric, that also handles repeated eigenvalues correctly.

Prompt 2

Q3 proof: do it with index notation, trace(AB) = sum_i sum_k A_ik B_ki, swap the sums. note it holds for A m×n, B n×m, not only square.

Q4: det(C) = 24-14 = 10 ≠ 0 so it's invertible, then the 2x2 adjugate formula. show C C^-1 = I. for the function use np.linalg.inv on a float copy, don't do the adjugate in code since it has to work for nxn.

Prompt 3

part 2. Q1 is just poisson with rate 3, so c = e^-3 from the exp taylor series, and E[X] = Var(X) = 3. show the derivation though, shift the index for E[X] and use E[X(X-1)] for the variance, don't just cite poisson. function generalizes to any rate.

Q2: P(A∩B) = P(A|B)P(B) = .18, Bayes for P(B|A) = .36, inclusion-exclusion for the union = .62.

Prompt 4

Q3 coin: likelihood θ^h(1-θ)^(n-h), log, derivative, θ_MLE = h/n = 3/5. for MAP the prior 2θ just adds one to the exponent so the posterior is ∝ θ^(h+1)(1-θ)^(n-h), mode (h+1)/(n+1) = 4/6 = 2/3. docstring says include boundary maxima, so check h=0 and h=n explicitly. MLE at h=0 is 0 and at h=n is 1, where the derivative argument breaks. make sure the code returns the right thing there and doesn't divide by zero.

Prompt 5

Q4 gaussian MLE: derive both partials of the log likelihood. mean 4, variance uses 1/n not n-1 so it's 5. use np.var with ddof=0.

Q5: posterior over μ is gaussian, so the mode is the precision-weighted average $(x/\sigma^2 + \mu_0/\tau^2)/(1/\sigma^2 + 1/\tau^2)$. with x=5, σ^2=4, prior N(0,1) that gives 1. derive it by setting the derivative of the log posterior to zero. remember the args are variances, not std devs.

Q6: expand E[(X-μ)^2] with linearity of expectation.

Prompt 6

part 3. Q1: define Σ = E[(X-μ)(X-μ)^T], then Var(u^T X) = E[(u^T(X-μ))^2] = u^T Σ u. write each step.

Q2: generate_data with np.random.default_rng(seed).multivariate_normal, default mean [2,-1] and cov [[3,1.2],[1.2,1]]. comment on the correlation, ρ = 1.2/sqrt(3) ≈ 0.69, so a clear positive tilt.

Q3 pca: copy X as float, center, cov with divisor n-1 (np.cov rowvar=False or by hand), eigh, then flip to descending order since eigh returns ascending. components are columns, scores = centered @ components. eigh keeps it orthonormal even in tied or rank-deficient cases. clip tiny negative eigenvalues to 0 from float error. return the dict with the exact keys from the docstring. connect every step back to the Q1 derivation in the comments: first PC = argmax of u^T Σ u subject to ||u||=1, which is the Lagrangian eigenvalue problem.

Prompt 7

Q4 explained_variance = eigenvalues / sum, same shape. discussion: with this covariance PC1 should carry roughly 88-90%, so projecting to 1D keeps most of the variance. explain the tradeoff (compression and denoising vs info lost along PC2).

make_plots: return a figure with two subplots. left is the scatter with both PC directions drawn from the mean, scaled by sqrt(eigenvalue). right is a bar chart of the explained variance fractions. label the axes and add a legend, no show/save inside the function. main block: generate, fit, print all the part 1/2 results, save the figure to pca.png. fill in the reflection section too.

Prompt 8

give me the full final Assignment1.py in one block. then a quick test script that checks: defaults match the hand answers, pca on a rank-deficient X (second column = 2*first) and an isotropic X, coin_estimates(0,5) and (5,5), and that no function mutates its input arrays.

Prompt 9

now package everything for canvas so i can just upload it. give me:

1. the final Assignment1.py

2. a typed PDF of all the derivations, every question from parts 1-3 in order, with the math formatted properly (not code comments) and the final answers clearly marked

3. pca.png from running the main block, also embedded in the PDF under part 3 next to the Q2/Q3/Q4 discussion

4. a zip with all three files

actually run the code to make the plot and check the numbers in the PDF match what the functions return.